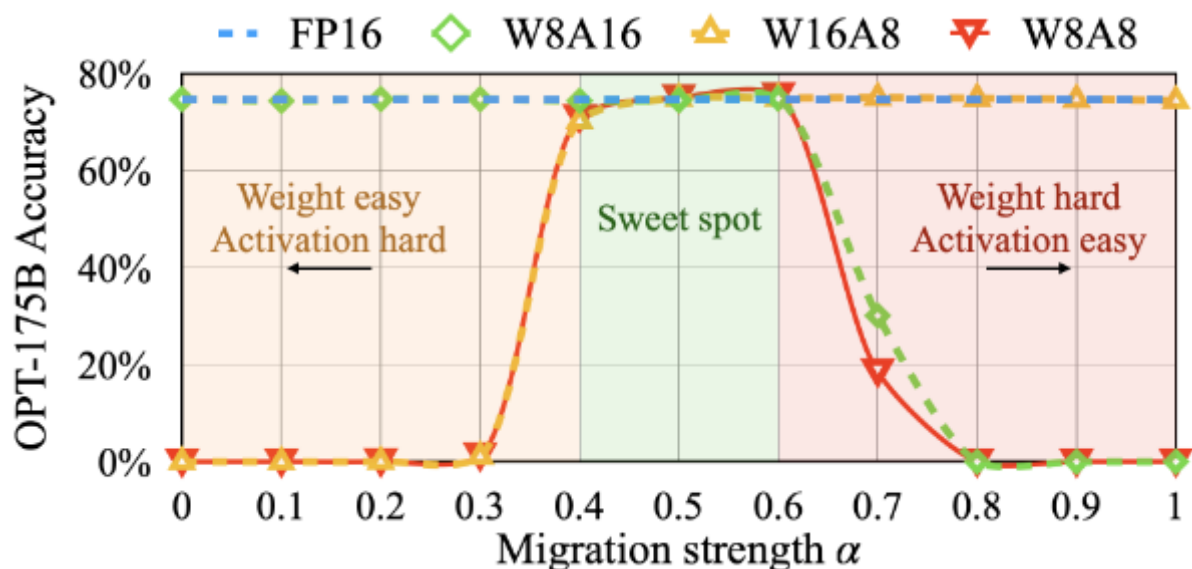




- α 太小 $\rightarrow$ 激活难量化 $\rightarrow$ 精度下降
- α 太大 $\rightarrow$ 权重难量化 $\rightarrow$ 精度下降
- $\alpha \approx 0.5 \rightarrow$ 通常是最优平衡点 (sweet spot)

其他细节

关于为什么SmoothQuant通常采取W8A8的量化策略，而不太常用W4A16等量化策略：

一个可能的原因，请辩证看待：可能因为“Smooth”的方法将离群值进行“转移”，比较理想的状态是激活值与权重的分布基本均衡，效果才会最佳（这与paper中发现最佳的smoothing_factor是基本一致的），因此对激活值和权重使用相同的8bit量化精度是合适的。

2. AWQ

3. GPTQ

4. GPTVQ

5. VPTQ

6. Q-VLM

这篇论文提出了一种新颖的后训练量化框架，名为**Q-VLM**，旨在加速大型视觉-语言模型（LVLMs）的推理，提升多模态推理效率。与传统方法不同，**Q-VLM**不只是逐层优化量化，而是通过挖掘层间依赖

性来有效地寻找最优的量化策略。具体来说，Q-VLM关注输出的离散化误差在不同层间的相互依赖，利用这种依赖关系来减少量化噪声，从而提高模型的性能。

论文创新点/为了解决的问题：

- 挖掘层间依赖：**传统方法在进行量化时忽略了不同层之间的依赖关系，而Q-VLM通过优化每一层的量化方案，减少量化误差，从而提升整体性能。
- 使用激活熵作为代理）：**为了高效地缩小搜索空间，Q-VLM引入了激活熵这一指标，作为量化函数搜索的代理，从而使得量化过程更加高效。

Q-VLM 论文的创新点主要集中在量化策略的优化，而不是量化算法本身的根本改变。具体来说，Q-VLM的创新体现在以下几个方面：

1. 量化策略的优化：

- 熵基搜索：**Q-VLM使用熵来评估每层激活值的分布，动态调整量化范围。这使得Q-VLM在量化过程中更加灵活，根据每一层激活的“波动性”调整量化范围，从而保持量化精度。这个策略在很多传统量化方法中并未采用，传统方法通常是固定的量化范围。
- 跨层依赖性优化：**传统的量化方法通常逐层独立量化，每一层的量化参数（如最小值和最大值）是独立计算的。而Q-VLM通过挖掘层与层之间的依赖关系，联合优化多个层的量化参数，避免了逐层量化带来的误差积累。通过分析层间的依赖性，Q-VLM优化了整体模型的量化效果。

2. 激活量化和权重量化的统一：

- Q-VLM不仅对**权重**进行量化，还同时对**激活值**进行量化，这一点和很多传统量化方法不同。大部分量化方法只对权重进行量化，激活值通常不进行量化，保持较高的精度。Q-VLM通过对激活值和权重同时进行量化（如W4A4量化），进一步减少了内存占用，并加速了推理过程。

3. 动态量化范围调整：

- Q-VLM通过**动态调整量化范围**，确保量化后的模型精度尽可能高。这个调整不仅仅是通过固定的公式，而是基于当前层激活值的分布和变化（通过熵的计算），使得量化范围能够适应不同层的特性。

量化算法的具体实现：

尽管Q-VLM对量化策略进行了优化，但其基础量化算法（基于**bitsandbytes**库实现）并没有根本改变。换句话说，Q-VLM仍然使用传统的量化方法（）来执行量化操作，而创新点在于如何**灵活地调整量化过程中的参数**（如最小值、最大值、scale、zero-point）以适应不同层的特性。

总结：

Q-VLM的创新主要体现在**量化策略的优化**上，尤其是通过**熵基搜索**、**跨层依赖性优化**以及**激活值和权重同时量化**，来提高量化精度并减少计算开销。这些策略使得Q-VLM能够在保持高准确度的同时，大

幅度减少模型的存储需求和计算成本。

一些理解：

1. 挖掘层间依赖性来有效地寻找最优的量化策略：

- 传统量化方法的缺点是它们没有考虑各层之间可能存在的关联。它们会逐个处理每一层的量化，通过最小化误差来调整每层的量化方式，但这种方法没有充分挖掘层与层之间的相互影响。因此，它可能无法找到最优的量化策略。
- 而在这篇论文中，**Q-VLM**引入了**层间依赖性**，即量化不仅仅是每层单独优化，而是考虑层与层之间的关系，综合优化多个层的量化效果。这样能够得到更好的整体量化策略。

2. 离散化误差在不同层间的相互依赖：

- 量化时，模型的权重和激活值会被转化为较低精度的数字，这个过程中会引入误差（称为离散化误差）。在传统方法中，量化误差通常是独立考虑的，但实际上，每一层的误差会影响到后续层的输出，进而影响整个模型的性能。Q-VLM通过**考虑层之间的误差关系**，优化这些误差的传播，减少量化带来的整体噪声（误差），从而提升量化模型的表现。

3. 激活熵作为代理

- 想象你在做量化时，网络每一层的计算结果（叫做激活值）就像是一个个数字。这些数字可以大，也可以小，变化很大，或者变化不大。
- **激活熵**就是用来衡量这些数字变化的“复杂度”或“混乱度”。如果某一层的数字变化很大，那我们就说它的熵高，说明这一层的数值很复杂，量化起来可能会有更多误差。反之，熵低的层，变化小，量化误差就比较小。
- **代理**的意思是，Q-VLM通过计算熵来预测哪些层容易出错（即熵高的层），然后调整量化策略，避免量化过程引入太大误差。

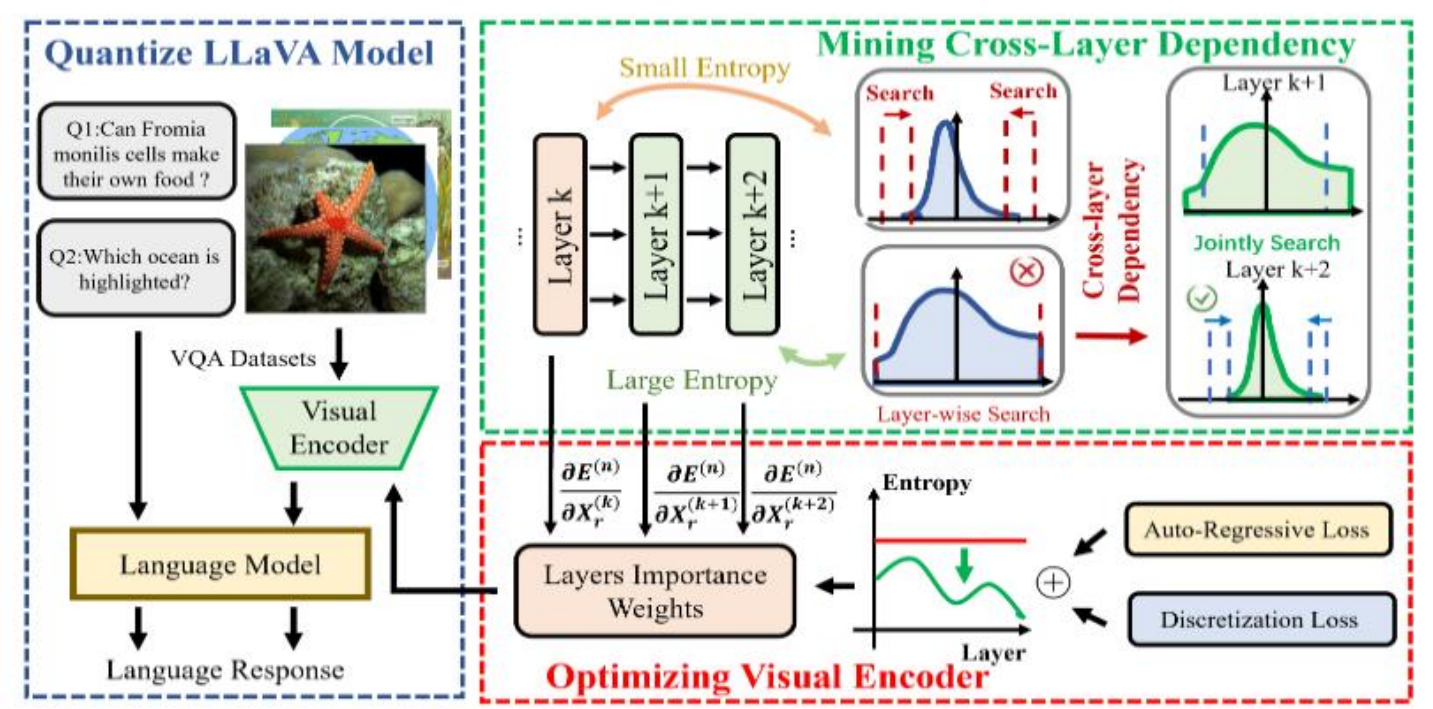
4. 量化搜索空间

- 量化就是让模型更小、更快，但是为了实现这一点，我们需要找到适合每一层的“压缩”方法。比如，是用8位整数来表示，还是用更低的位数来表示？
- 这些“压缩方法”就是量化参数，而这些参数可以有很多种组合，形成一个**搜索空间**。
- 量化的目的是在这个大空间里找到最适合的参数组合，使得压缩后的模型还能保持较好的表现，同时减少计算和存储的开销。

5. 为什么要进行量化搜索？

- 每一层的数字（激活值）变化不一样，所以需要不同的量化策略。比如，有些层的激活值变化不大，可以用较低的精度；而有些层则变化大，可能需要更高的精度来保留信息。
- 量化搜索就是为了找到最合适的量化策略，让每一层都能够用最合适的精度来存储，从而减少误差并提升模型效率。

Q-VLM量化流程：



1. 量化LLaVA模型（左侧）：

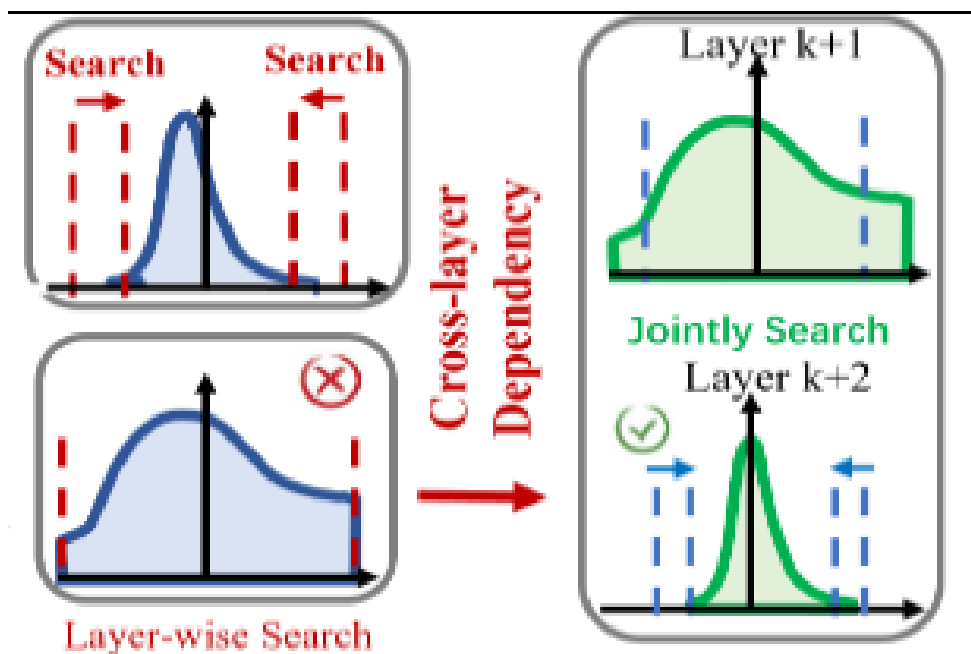
- 在这部分，模型的视觉编码器（负责处理图像信息）和语言模型（负责处理文本信息）一起工作。通过视觉问答（VQA）数据集，模型被训练用来回答图像相关的问题（如“Can Fromia monilis cells make their own food?”）。
- 视觉编码器生成与输入图像相关的特征，语言模型则根据这些特征和问题生成语言回答。

2. 引入层间依赖（绿色框）：

- 图中的绿色部分展示了层间依赖性挖掘的过程。模型根据不同层的熵（信息量的多寡）来确定如何优化每一层的量化策略。
- 在小熵的层中（激活变化较小），模型会选择独立量化。对于大熵的层（激活变化大），模型需要进行联合搜索，即考虑这些层之间的相互依赖，优化整个网络的量化方式。

3. 优化视觉编码器（红色框）：

- a. 在这个部分，模型通过优化视觉编码器来调整量化策略。优化过程包括：
 - 自动回归损失（Auto-Regressive Loss）：确保量化后模型能保持合理的顺序或结构。
 - 离散化损失（Discretization Loss）：通过对量化后数据的误差进行最小化，提升整体量化效果。
- b. 同时，模型也在进行层的权重重要性评估，即判断哪些层在量化过程中最重要，确保它们得到更精确的处理。



1. Layer-wise Search（逐层搜索）

- 图中的**红色框**表示的是逐层搜索。在这种方法中，每一层的量化参数是独立选择的。每一层的量化精度是根据该层的熵值（即信息量的多少）单独优化的。
- 这种方法的问题是，层与层之间的依赖关系没有被考虑到。每一层都单独进行优化，因此可能会导致整体量化效果不理想，忽略了层间的相互影响。

2. Cross-layer Dependency（层间依赖）

- 图中的**绿色框**展示了层间依赖的情况。通过引入**跨层依赖性**，Q-VLM在量化时不仅考虑单独每一层，而是考虑多个层之间的关系。
- 联合搜索（Jointly Search）**表示在量化过程中，多个层（比如Layer k+1 和 Layer k+2）会被一起优化，考虑它们之间的相互依赖。通过这种方式，可以优化整体量化效果，使得不同层之间的误差影响最小化，提升整体模型的精度。

具体算法：

公式（1）：

优化目标：

$$\begin{aligned} \min_{\{Q_k\}} \quad & J = \left\| W_q^{(n)} X_q^{(n)} - W_r^{(n)} X_r^{(n)} \right\|_2^2 \\ \text{s.t.} \quad & X_q^{(k+1)} = Q_k(W_q^{(k)} X_q^{(k)}) \end{aligned}$$

- **作用：**这个公式定义了量化过程中的损失函数。目标是通过调整量化的权重和激活值，最小化量化后的输出与全精度模型输出之间的误差。这里， $W_q^{(k)}$ 和 $X_q^{(k)}$ 表示第 k 层的量化权重和激活值，而 $W_r^{(k)}$ 和 $X_r^{(k)}$ 是对应的全精度值。 Q_k 是第 k 层的量化函数。

公式（2）：

逐层搜索的目标：

$$\min_{Q_k} J = \left\| W_q^{(k)} X_q^{(k)} - W_r^{(k)} X_r^{(k)} \right\|_2^2$$

- **作用：**这个公式描述了传统的量化方法，其中每一层的量化函数是独立搜索的。其目标是通过最小化每层量化后的误差来进行优化，但这种方法没有考虑跨层的依赖关系，因此可能导致整体模型量化效果不佳。

公式（3）（关键公式）：

块级量化的优化目标：

$$\begin{aligned} \min_{\{Q_k\} \in B_i} J &= \left\| W_q^{(L_i)} X_q^{(L_i)} - W_r^{(L_i)} X_r^{(L_i)} \right\|_2^2 \\ s.t. \quad X_q^{(k+1)} &= Q_k(W_q^{(k)} X_q^{(k)}) \end{aligned}$$

- **作用：**为了避免逐层搜索的局限性，Q-VLM将模型划分为多个块（block），并在每个块中进行量化。这个公式的目标是通过最小化每个块内的量化误差来优化量化函数。块内的量化函数通过考虑多个层的输出误差来提高整体优化效果。
- 通过将模型划分为多个块，优化每个块内多个层的量化策略，减少了逐层独立量化的低效性，并有效降低了搜索成本。

公式（4）（关键公式）：

层间依赖性（Cross-layer Dependency）：

用来衡量两个连续层之间的依赖关系的公式

$$D(k, k+1) = - \sum_{ij} p(x_{q,ij}^{(k)}, x_{q,ij}^{(k+1)}) \log p(x_{q,ij}^{(k+1)} | x_{q,ij}^{(k)})$$

- **作用：**这个公式定义了层与层之间的依赖性，计算了第 k 层和第 k+1 层激活值的离散化误差差异（Discretization Error Difference, DED）。这个值用于衡量量化误差在层与层之间的传播。依赖性越强，量化误差在后续层的影响越大，因此需要联合优化。

公式（5）：

量化分布的近似：

这个公式用于计算量化激活值的概率分布

$$p(x_{q,ij}^{(k)}) = \frac{\exp(-(x_{r,ij}^{(k)} - q_m)^2 / \Delta)}{\sum_{m=1}^M \exp(-(x_{r,ij}^{(k)} - q_m)^2 / \Delta)} \cdot \delta(x_{q,ij}^{(k)} - q_m)$$

- **作用：** 这个公式描述了如何通过近似方法计算量化过程中每个激活值的分布。在量化过程中，激活值会被映射到量化点 q_m ，这里的分布通过高斯函数和标准化因子来描述， δ 是脉冲分布函数。

公式（6）：

跨层依赖的扩展：

这个公式用于评估非连续层（如层 k_r 和层 k_s 之间的跨层依赖性。它将多个中间层的熵加在一起，从而表示层间的依赖。

$$D(k_r, k_s) = - \sum_{k=k_r}^{k_s} \sum_{ij} p(x_{q,ij}^{(k)}, x_{q,ij}^{(k+1)}) \log(x_{q,ij}^{(k+1)} | x_{q,ij}^{(k)})$$

- **作用：** 这个公式扩展了跨层依赖的计算，表示非连续层之间的依赖性。通过计算跨层的依赖性，Q-VLM进一步提高了量化的精度和效率，避免了传统方法中的量化误差积累问题。

公式（7）：

块的划分：

$$B_i = \{ \bigcup_{k=k_r}^{k_s} Q_k | D(k_r, k_s) > (k_s - k_r)h_0 \}$$

- **作用：** 这个公式描述了如何根据跨层依赖性划分模型块。如果两个层之间的跨层依赖性大于某个阈值 h_0 ，则这些层将被划分到同一个块中进行联合量化。这种划分方式有助于减少量化误差，并使得量化过程更加高效。

公式（8）（关键公式）：

视觉编码器优化的损失函数：

$$L_{ent} = \sum_{k=1}^n \left\| \frac{\partial E^{(n)}}{\partial X_r^{(k)}} \right\| \cdot \sum_{ij} p(x_{q,ij}^{(k)}, x_{q,ij}^{(k+1)}) \log p(x_{q,ij}^{(k+1)} | x_{q,ij}^{(k)})$$

- **作用：**这个公式描述了优化视觉编码器时所使用的损失函数。它考虑了量化误差对最终输出的影响，并通过计算每一层的雅可比矩阵，来评估该层对量化误差的贡献，从而帮助优化量化过程。

公式（9）（关键公式）：

视觉编码器和整体量化误差的加权损失：

$$L_{err} = \|X_q^v - X_r^v\| + \eta \|X_q^{(n)} - X_r^{(n)}\|$$

- **作用：**这个公式是视觉编码器优化的目标函数之一，旨在最小化视觉编码器和整个模型的量化误差。它结合了视觉编码器和后续层的量化误差，其中值 η 是平衡两个误差项的超参数。

公式（10）：

视觉编码器的整体优化目标：

$$L = L_{reg} + \lambda_1 L_{ent} + \lambda_2 L_{err}$$

- **作用：**这是视觉编码器优化的整体目标函数，它包括了正则化损失（ L_{reg} ）、熵最小化损失（ L_{ent} ）和量化误差损失（ L_{err} ）。通过优化这个目标函数，Q-VLM能够优化视觉编码器，从而有效地减少量化过程中的误差。

这些公式的共同目的是通过引入层间依赖性和优化视觉编码器，提升量化精度并减少搜索成本，从而实现高效的后训练量化。

实验效果：

1. 实验设置：

- 作者使用了LLaVA和MoE-LLaVA模型，并在不同的量化位宽设置下进行了评估。量化位宽（bitwidth）设置为6位和4位，旨在测试Q-VLM在不同压缩率下的效果。
- 评估任务主要是**多模态问答**（如ScienceQA数据集）等推理任务，验证Q-VLM在处理视觉和文本结合的多模态任务时的表现。

2. 量化效果评估：

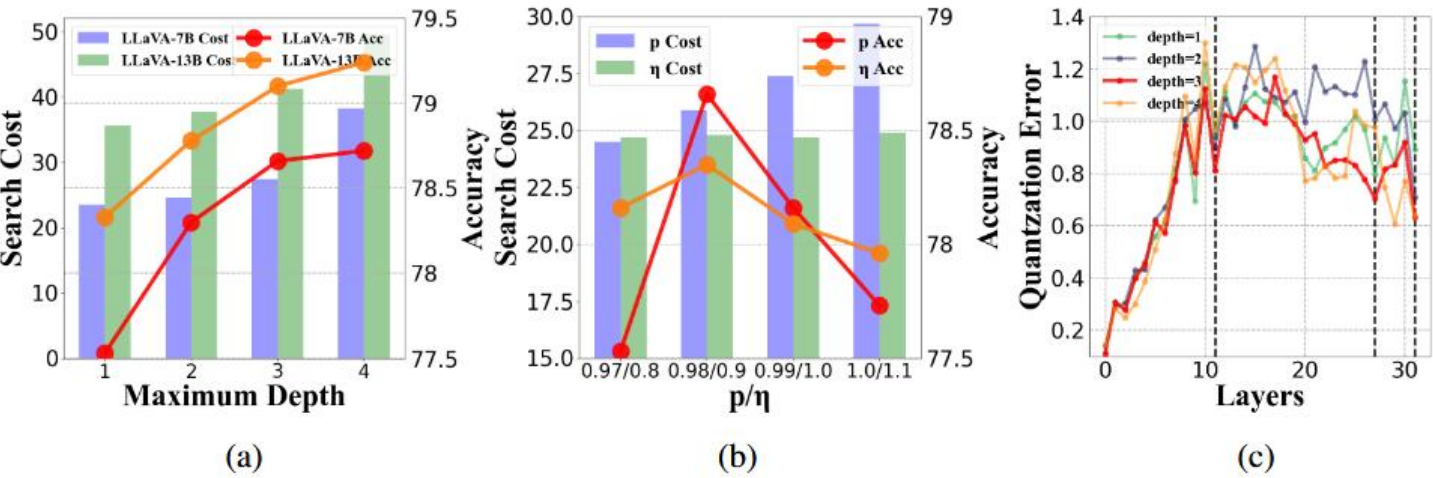
Method	W6A6			W4A4		
	Memory	Search cost	Accuracy	Memory	Search cost	Accuracy
QLoRA	9.7G	23.7	88.43	6.6G	23.5	77.53
+CDM		26.9	88.95		25.9	78.66
+VEO		24.1	88.72		23.7	78.35
Q-VLM		25.1	89.34		24.6	79.79

Table 1: Effect of different LVLM quantization method we proposed. "CDM" means cross-layer dependency mining and "VEO" stands for visual encoder optimization. We report the result of LLaVA-7B model on ScienceQA dataset.

展示了不同量化方法（Q-VLM、QLoRA、CDM、VEO）对LLaVA-7B模型在ScienceQA数据集上的效果。

结果：

- **Q-VLM**在6位量化（W6A6）下，内存消耗为25.1GB，搜索成本为24.6，准确率为89.34%。
- **Q-VLM**在4位量化（W4A4）下，内存消耗为24.6GB，搜索成本为23.7，准确率为79.79%。
- 对比而言，**Q-VLM**在内存消耗和准确度上均优于**QLoRA**和其他方法，特别是在优化视觉编码器（VEO）和跨层依赖性挖掘（CDM）时，效果更加显著。



- **图 (a)**：显示了最大层深度对回答准确性和搜索成本的影响。随着最大层深度增加，**LLaVA-7B**的搜索成本增加，但其准确度保持稳定。深度为3时，性能最为平衡。
- **图 (b)**：显示了回答准确度和搜索成本相对于不同的超参数 p/η 的变化。通过调整这些超参数，可以在准确度和搜索成本之间找到平衡。
- **图 (c)**：展示了不同最大层深度对各层量化误差的影响。量化误差随着层数的增加而波动，但选择适当的层深度和超参数能够减少这些误差。

总结：Q-VLM相比其他方法在**内存消耗**和**准确度**上都表现出色，尤其是在优化视觉编码器和挖掘层间依赖性后，能够在减少计算开销的同时保持较高的准确度。

	Bits	Method	Subject			Context Modality			Average	
			NAT	SOC	LAN	TXT	IMG	NO		
LLaVA-7B	FP	-	89.39	96.06	85.64	88.71	87.65	88.50	89.81	
	W6A6	AWQ	85.39	92.01	83.27	84.80	83.54	85.99	86.23	
		QLoRA	88.45	94.71	84.45	87.63	86.07	87.87	88.43	
		Q-VLM	89.43	95.73	84.00	88.71	87.51	87.25	89.34	
	W4A4	AWQ	74.33	72.22	74.82	73.41	67.13	77.98	74.02	
		QLoRA	77.53	75.48	79.18	76.64	70.70	81.95	77.53	
		Q-VLM	80.86	75.93	80.73	80.01	72.48	83.90	79.79	
	LLaVA-13B	FP	-	90.19	93.14	87.09	89.39	87.06	89.83	90.00
		W6A6	AWQ	88.03	92.60	84.00	86.02	85.18	86.41	87.57
QLoRA			88.87	92.89	85.64	87.59	86.56	87.53	88.87	
Q-VLM			89.54	93.18	86.50	88.12	87.01	88.85	89.70	
W4A4		AWQ	80.71	70.61	78.49	79.46	70.76	81.82	77.91	
		QLoRA	79.62	71.43	82.45	78.25	68.42	85.30	78.64	
		Q-VLM	82.55	73.32	83.18	81.03	70.82	86.74	80.78	
MoE-LLaVA-1.6B		FP	-	64.01	58.57	63.30	62.80	54.78	66.97	62.68
		W6A6	AWQ	59.83	57.78	61.48	58.94	52.95	64.25	59.83
	QLoRA		62.98	57.78	61.85	61.87	53.99	65.37	61.60	
	Q-VLM		64.14	58.68	62.85	62.80	55.23	66.69	62.46	
	W4A4	AWQ	53.69	49.58	54.27	52.52	47.81	57.65	52.98	
		QLoRA	54.48	49.69	55.55	53.20	47.30	57.58	53.24	
		Q-VLM	55.06	51.94	56.27	54.57	48.03	58.34	54.72	

LLaVA-7B:

- 在 W6A6 位宽下，Q-VLM 的平均准确度为 **89.43%**，表现最优。
- 在 W4A4 位宽下，Q-VLM 的准确度为 **79.79%**，也领先于其他方法。

LLaVA-13B:

- 在 W6A6 位宽下，Q-VLM 达到了 **89.54%** 的准确度，超越了 AWQ 和 QLoRA。
- 在 W4A4 位宽下，Q-VLM 依然表现出色，达到了 **80.73%**。

MoE-LLaVA-1.6B:

- 在 W6A6 位宽下，Q-VLM 达到 **64.14%** 的准确度，显著高于 AWQ 和 QLoRA。
- 在 W4A4 位宽下，Q-VLM 的准确度为 **55.06%**，领先于其他方法。

Model	Dataset	FP	W6A6			W4A4		
			AWQ	QLoRA	Q-VLM	AWQ	QLoRA	Q-VLM
LLaVA-7B	SQA	66.79	65.87	66.16	66.67	56.73	56.50	57.70
	VizWiz	49.87	48.51	49.23	49.86	44.29	44.73	45.82
	VQA v2	78.50	77.51	77.58	78.52	71.89	72.02	72.51
LLaVA-13B	SQA	71.61	71.37	71.52	72.27	68.13	68.04	68.84
	VizWiz	53.63	52.35	52.85	53.69	47.55	47.97	49.28
	VQA v2	79.94	78.83	79.25	79.65	71.55	72.19	73.02

不同VQA数据集下的性能比较

在LLaVA-7B模型各个VQA数据集（SQA、VizWiz、VQA v2）上：

- Q-VLM在各个数据集上的表现相较于AWQ和QLoRA有了显著的提升，特别是在VQA v2数据集上，Q-VLM的准确度为 **78.52%**，优于其他方法。
- 在SQA和VizWiz数据集上，Q-VLM也超越了AWQ和QLoRA，显示了较强的多模态推理能力。

Method	FP			W8A8			W4A4		
	Time	Memory	Accuracy	Time	Memory	Accuracy	Time	Memory	Accuracy
QLoRA				16.7h	16.5G	89.32	17.0h	10.7G	78.64
AWQ	12.9h	24.0G	90.00	11.2h	17.2G	88.94	8.9h	11.2G	77.91
Q-VLM				11.2h	15.7G	89.82	8.9h	9.6G	80.78

在Science QA数据集上的时间、内存和准确度比较

该表格展示了Q-VLM、AWQ和QLoRA在LLaVA-13B模型下，在不同量化位宽下（FP、W8A8、W4A4）的时间、内存和准确度的对比。

- 在FP（全精度）下，Q-VLM的准确度为90.00%，时间为12.9小时，内存为24.0GB。
- 在W8A8位宽下，Q-VLM的准确度为89.82%，时间为16.7小时，内存为16.5GB。
- 在W4A4位宽下，Q-VLM的准确度为80.78%，时间为10.7小时，内存为10.7GB。

总结：

1. Q-VLM在所有量化位宽下都表现出色，特别是在W4A4位宽下，相比于AWQ和QLoRA，提供了较好的准确度和较低的内存消耗。
2. Q-VLM的性能在视觉问答任务中非常优越，尤其是在多个VQA数据集上的准确度提升，以及在Science QA数据集上的高效量化。
3. 对比时间和内存消耗，Q-VLM在保持较高准确度的同时，也优化了计算开销，尤其是在W4A4量化设置下，提供了良好的平衡。

Q-VLM 项目的背景、动机、框架与量化流程



Q-VLM-2410.08119.md

背景与动机

在Q-VLM论文中，作者提出了一个新的后训练量化（PTQ）框架，专门用于大型视觉语言模型（LVLMs）的高效推理。以下是论文的主要动机：

- 问题陈述：
 - 大型视觉语言模型（LLaVA等）在计算资源方面的要求很高，导致其在移动设备和机器人等资源受限的场景下无法部署。
 - 现有的量化方法通常逐层搜索量化函数，但这些方法没有考虑到跨层依赖性，这导致了误差的累积，从而影响模型的推理精度。
- 核心洞察：
 - 激活熵（Activation Entropy）与跨层依赖性之间有强烈的相关性。高熵层通常表示该层激活值的变化范围较大，量化误差也较大，影响后续层的量化精度。

Q-VLM 的核心创新点

Q-VLM的主要创新体现在以下几个方面：

- 跨层依赖性挖掘（Cross-layer Dependency Mining）：
 - 传统的量化方法逐层独立地优化量化函数，而Q-VLM通过考虑层与层之间的依赖关系来联合优化多个层的量化，从而减少量化误差。

2. 基于熵的量化策略 (Entropy-based Quantization) :

- Q-VLM使用**激活熵**来动态调整量化的范围。熵越大的层（即，激活值变化越大的层）需要更多的精度来减少量化误差，Q-VLM会将这些高熵层单独优化，而低熵层则合并在一起优化。

3. 优化视觉编码器:

- Q-VLM不仅优化了跨层依赖关系，还**优化视觉编码器**，通过解耦不同层之间的依赖，进一步减少量化的计算成本，并保持高精度。

Q-VLM的整体框架

Q-VLM的框架分为三个主要阶段:

- 激活熵分析 (Entropy Analysis)** : 通过前向传播计算每一层的激活熵。
- 块划分 (Block Partitioning)** : 基于熵值和层间依赖性，将模型划分为多个块，每个块包含多个层，减少搜索空间。
- 联合搜索 (Joint Search)** : 针对每个块，通过联合搜索量化函数来最小化量化误差。

Q-VLM的量化流程:

代码块

```
1 Step 1: 加载 全精度 模型
2 Step 2: 用校准集 forward (几十~几百条数据)
3 Step 3: 在这个过程中:
4     - 统计激活分布
5     - 计算熵
6     - 调整 min/max (quant_modules里的代码)
7     - 搜索最优量化参数
8 Step 4: 量化参数固定 (冻结)
9 Step 5: 正式推理 (不再改变)
```

1. 激活量化:

- `QuantAct` 类通过**线性量化**方法对激活值进行量化，并通过动态调整**量化范围**来减少量化误差。量化后的激活值被压缩到较低的精度（例如4-bit），减少内存占用。
- 熵基搜索**: Q-VLM通过计算每层的激活熵来动态调整量化的最小值和最大值。激活值的熵越高，表示该层的激活变化范围越大，需要更精细的量化。量化范围根据这些熵值动态更新。

a. 代码实现 (`quantization` 函数) :

代码块

```
1 def quantization(self, inputs, quantization_min, quantization_max):
2     # 步骤 1: 获取量化参数
```

```

3      # 使用 `asymmetric_linear_quantization_params` 函数来计算量化的 scale 和
      zero_point
4      # scale: 量化的比例, zero_point: 量化的偏移量
5      scale, zero_point = asymmetric_linear_quantization_params(
6          self.activation_bit, quantization_min, quantization_max
7      )
8
9      # 步骤 2: 量化过程
10     # 将激活值输入按比例缩放并减去偏移量后进行四舍五入, 得到量化的整数值
11     new_quant_x = torch.round(scale * inputs - zero_point)
12
13     # 步骤 3: 用于量化误差校正的步骤
14     # 这里通过修正量化后的值来减小量化误差。通过计算两个边界 (-new_quant_x 和
      new_quant_x - (n - 1)) 的绝对值差异来进行调整。
15     n = 2**(self.activation_bit - 1) # 用于获取量化的步长, 计算激活值的量化步长
16     new_quant_x_1 = 0.5 * ((-new_quant_x - n).abs() - (new_quant_x - (n -
17     1)).abs() - 1)
18
19     # 步骤 4: 将量化值反量化
20     # 通过将量化后的值加上零点偏移并按比例缩放, 得到调整后的量化激活值
21     quant_act = (new_quant_x_1 + zero_point) / scale
22
23     # 返回量化后的激活值
24     return quant_act

```

2. 动态量化范围调整:

- 根据激活值的最小值和最大值, Q-VLM会动态调整量化的最小值和最大值(即量化范围)。如果某一层的熵较高, Q-VLM会加大量化的范围, 从而减少量化误差。

a. 代码实现(调整量化范围):

代码块

```

1  # 更新量化最小值 (llama_range_min)
2  # 如果 x_min 比 self.llama_range_min 更小, 意味着当前激活值的最小值更小, 需要更新
      llama_range_min。
3  self.llama_range_min += -self.llama_range_min +
      torch.min(self.llama_range_min, x_min)
4  # 更新量化最大值 (llama_range_max)
5  # 如果 x_max 比 self.llama_range_max 更大, 意味着当前激活值的最大值更大, 需要更新
      llama_range_max。
6  self.llama_range_max += -self.llama_range_max +
      torch.max(self.llama_range_max, x_max)

```

3. 联合优化和搜索:

- 通过**跨层依赖性分析**，Q-VLM将模型分成多个块，并在每个块内联合优化量化参数。量化的最小值和最大值在每个块内进行优化，以减少跨层依赖带来的误差。

a. 代码实现（跨层依赖性计算）：

该代码计算了两层激活值之间的依赖关系，并返回了一个**依赖性度量**（DED），用于联合优化量化过程。

在Q-VLM中，层间**依赖性强**并没有明确的**阈值**，它是通过计算**相对熵**和**条件熵**等度量值来表示的。**阈值**是一个动态的值，通常依赖于模型的训练情况和量化的具体策略。

代码块

```
1  def compute_DED(self, p_k, p_k1):
2      p_k = F.normalize(p_k, p=1, dim=1) # 对当前层的激活进行归一化
3      p_k1 = F.normalize(p_k1, p=1, dim=1) # 对下一层的激活进行归一化
4
5      # 计算两层激活值的联合分布
6      joint_p = p_k * p_k1
7      joint_p = joint_p / joint_p.sum(dim=1, keepdim=True) # 归一化联合分布
8
9      # 计算条件分布  $p(k+1) | p(k)$ 
10     condition_p = p_k1 / (p_k + 1e-5)
11     condition_p = condition_p / condition_p.sum(dim=1, keepdim=True) # 归一化条件分布
12
13     # 返回两层之间的依赖性度量 (DED)
14     return -1 * torch.sum(joint_p * torch.log(condition_p + 1e-5),
                             dim=1).mean()
```

Q-VLM 是动态量化还是静态量化？

Q-VLM采用的是**动态量化**方法，因为它在**用校准集推理阶段动态地调整量化范围**，并基于每层激活的熵计算量化范围，量化的参数（如最小值和最大值）是根据数据动态调整的，而不是固定的。相对于传统的静态量化方法，动态量化能够更灵活地适应模型中各层激活的变化，避免了固定量化范围可能带来的精度损失。

核心创新点和量化流程总结

- 核心创新**：Q-VLM的核心创新是**跨层依赖性挖掘**和**基于熵的动态量化范围调整**，这两者使得Q-VLM能够显著减少量化误差，并加速推理过程，同时保持高精度。
- 量化流程**：Q-VLM首先通过**熵分析**计算每层的激活熵，然后根据熵值动态调整量化范围，最后通过**联合搜索**对每个块的量化参数进行优化，减少误差积累。

项目的量化策略是动态量化，Q-VLM的核心贡献是在动态量化中引入了熵基搜索和跨层依赖性分析，从而更高效地进行量化并保持较高的准确性。